

POLITECHNIKA ŁÓDZKA

WYDZIAŁ FIZYKI TECHNICZNEJ, INFORMATYKI
I MATEMATYKI STOSOWANEJ

Kierunek: Matematyka stosowana

PROJEKT "STABILNE SKOJARZENIA"

Maja Kmiecik 263307

ŁÓDŹ, CZERWIEC 2026

1 Wprowadzenie do problemu

Problem stabilnych skojarzeń (znany także jako **problem stabilnych małżeństw**) to klasyczny algorytm optymalizacyjny i koncepcja z teorii gier. Polega na połączeniu w pary elementów z dwóch różnych zbiorów na podstawie ich indywidualnych list preferencji.

Celem algorytmu jest znalezienie stabilnego dopasowania między dwoma rozłącznymi zbiorami (np. kobiet i mężczyzn) o jednakowych rozmiarach, przy ustalonej sekwencji preferencji dla każdego elementu. Dopasowanie jest odwzorowaniem (bijekcją) elementów z jednego zestawu do elementów drugiego zestawu. Rozwiązanie gwarantuje, że w żadnym układzie dwie osoby nie będą wołały siebie nawzajem od swoich obecnych partnerów. Dopasowanie nie jest stabilne, jeśli:

- Istnieje element A pierwszego dopasowanego zestawu, który preferuje dany element B drugiego dopasowanego zestawu nad elementem, do którego A jest już dopasowany, oraz
- B również woli A od elementu, do którego B jest już dopasowane.

Innymi słowy, dopasowanie jest stabilne, gdy nie pozostawi elementów po przeciwnych stronach, które nie są do siebie dobrane, ale wołałyby być.

Problem trwałego małżeństwa został określony w następujący sposób: Biorąc pod uwagę n mężczyzn i kobiet, gdzie każda osoba uszeregowała wszystkich członków płci przeciwnej w określonej sekwencji preferencji, dopasuj w pary małżeństw kobiety i mężczyzn, tak aby nie było dwóch osób przeciwnej płci, które wołałyby mieć siebie nawzajem zamiast obecnych partnerów. Gdy nie ma takich par, zbiór małżeństw uznaje się za stabilny.

2 Zastosowania

Algorytm znajdowania rozwiązań problemu stabilnego małżeństwa ma szerokie zastosowanie w praktyce w tzw. rynkach dwustronnych, gdzie nie można użyć zwykłych mechanizmów rynkowych opartych na cenie, między innymi:

- **Edukacja:** Przydzielanie uczniów do szkół lub studentów do uniwersytetów i akademików. System bierze pod uwagę preferencje kandydatów oraz wyniki egzaminów czy kryteria danej placówki, eliminując problem blokowania miejsc.
- **Medycyna:** Dopasowywanie absolwentów medycyny do szpitali na staże rezydentur (np. system NRMP w USA). Algorytm stosuje się również w programach przeszczepów łącząc pary dawca-biorca w optymalne łańcuchy.

- **Sieci CDN i infrastruktura internetowa:** Przydzielanie miliardów użytkowników do setek tysięcy serwerów na całym świecie (np. w usługach streamingowych czy stronach WWW). Użytkownicy są kojarzeni z najbliższymi serwerami (krótki czas odpowiedzi), a serwery z użytkownikami, których obsługa generuje najniższe koszty.
- **Aplikacje randkowe:** Łączenie użytkowników w pary na podstawie ich wzajemnych preferencji, lokalizacji i zainteresowań. Algorytm dba o to, aby rekomendować osoby, które mają największą szansę na obustronne polubienie się, zamiast pokazywać wszystkich wszystkim.

3 Rozwiązanie problemu

Do rozwiązania tego zadania służy algorytm Gale’a-Shapleya. W 1962 r. David Gale i Lloyd Shapley udowodnili, że dla każdej równej liczby mężczyzn i kobiet zawsze można rozwiązać SMP i zapewnić stabilność wszystkich małżeństw. Przedstawili algorytm, zwany również algorytmem odroczonej akceptacji (deferred acceptance algorithm), którego koncepcja wygląda następująco.

1. **"Oświadczyńy":** Każda osoba z grupy inicjującej (np. mężczyźni) oświadcza się osobie z grupy przeciwnej, którą najbardziej preferuje ze swojej listy (znajduje się na najwyższej pozycji w jego rankingu).
2. **Odrzucenie lub przyjęcie:** Gdy cała grupa inicjująca złożyła propozycje swoim wybrankom każda osoba z grupy przyjmującej (np. kobiety) tymczasowo zatrzymuje kandydata, którego woli najbardziej, a pozostałych odrzuca. Nie oznacza to jednak jeszcze akceptacji przez kobietę tego kandydata, chyba że jest to również mężczyzna najbardziej preferowany spośród wszystkich potencjalnych kandydatów.
3. **Kolejne propozycje:** Osoby odrzucone w poprzedniej rundzie oświadcza się kolejnym osobom ze swojej listy preferencji (z wyjątkiem tych, które już je odrzuciły).
4. **Zakończenie:** Proces trwa tak długo, aż wszyscy zostaną dopasowani. Algorytm zawsze się kończy i zawsze znajduje stabilne dopasowanie.

Co ważne, może istnieć wiele różnych stabilnych dopasowań. Grupa, która rozpoczyna oświadczyńy, otrzymuje najlepsze możliwe dopasowanie ze swoich preferencji, natomiast grupa przyjmująca otrzymuje dopasowanie najgorsze z możliwych.

4 Implementacja w Pythonie

Poniższy przykład pokazuje działanie algorytmu dobierając w stabilne pary grupy mężczyzn i kobiet. W skład grupy mężczyzn wchodzi Adam, Bartek oraz Cezary, a w skład grupy kobiet Ania, Kasia i Ola. Mają oni następujące preferencje (uszeregowane malejąco):

```
men_preferences = {
    "Adam": ["Kasia", "Ania", "Ola"],
    "Bartek": ["Ania", "Kasia", "Ola"],
    "Cezary": ["Ania", "Ola", "Kasia"]
}

women_preferences = {
    "Ania": ["Bartek", "Adam", "Cezary"],
    "Kasia": ["Adam", "Cezary", "Bartek"],
    "Ola": ["Adam", "Bartek", "Cezary"]
}
```

W celu uzyskania rozwiązania zaczynamy od zaimportowania szybkiej kolejki deque.

```
from collections import deque
```

Następnie definiujemy funkcję `stable_matching`, która będzie zwracała rozwiązanie problemu.

```
def stable_matching(men_preferences, women_preferences):
    # Lista wszystkich wolnych mężczyzn
    free_men = deque(men_preferences.keys())

    # Zapamiętuje aktualne zaręczyny
    engagements = {}

    # Zapamiętuje której kobiecie mężczyzna już się oświadczył
    proposals = {man: 0 for man in men_preferences}

    # Tworzymy ranking kobiet
    # (dzięki temu szybko sprawdzimy którego mężczyznę kobieta woli bardziej)
    women_ranking = {}
```

```

for woman, prefs in women_preferences.items():
    ranking = {}
    for rank, man in enumerate(prefs):
        ranking[man] = rank
    women_ranking[woman] = ranking

# Główna pętla algorytmu
while free_men:
    # Bierzemy wolnego mężczyznę
    man = free_men.popleft()

    # Wybiera najwyżej ocenianą kobietę, której jeszcze się nie oświadczył
    woman = men_preferences[man][proposals[man]]

    # Zwiększamy licznik oświadczyń
    proposals[man] += 1

    print(f"{man} oświadcza się {woman}")

    # Jeśli kobieta jest wolna
    if woman not in engagements:
        engagements[woman] = man
        print(f"{woman} akceptuje (była wolna)\n")

    else:
        current_partner = engagements[woman]

        # Sprawdzamy którego partnera kobieta woli bardziej
        if women_ranking[woman][man] < women_ranking[woman][current_partner]:
            print(f"{woman} woli {man} zamiast {current_partner}")

            # Stary partner staje się wolny
            free_men.append(current_partner)

            # Nowe zaręczyny
            engagements[woman] = man

```

```

        print(f"{current_partner} zostaje odrzucony\n")

    else:
        print(f"{woman} odrzuca {man}\n")

        # Mężczyzna dalej jest wolny
        free_men.append(man)

    return engagements

```

Gotową funkcję uruchamiamy dla naszych danych.

```

result = stable_matching(men_preferences, women_preferences)

print("STABILNE SKOJARZENIE:")
for woman, man in result.items():
    print(f"{man} + {woman}")

```

Otrzymujemy następujące wyniki.

Adam oświadcza się Kasia
Kasia akceptuje (była wolna)

Bartek oświadcza się Ania
Ania akceptuje (była wolna)

Cezary oświadcza się Ania
Ania odrzuca Cezary

Cezary oświadcza się Ola
Ola akceptuje (była wolna)

STABILNE SKOJARZENIE:
Adam + Kasia
Bartek + Ania
Cezary + Ola